

CODE REFERENCE

Data Pipeline — Line-by-Line Walkthrough

scripts/pipeline/ · raw CapMetro file to cleaned study set with weather

Group B3 · UST Electronics Engineering
An Evaluation of Multi-Agent Reinforcement Learning for
Dynamic Bus Scheduling Under Non-Ideal Conditions
Adviser: Asst. Prof. Kanny Krizzy D. Serrano, MSc

Code walkthrough — `scripts/pipeline/`

Line-by-line explanation of the data pipeline: raw CapMetro file → cleaned study set → weather attached. This is the code behind everything MSA1 presents.

Generated with `docs/prompts/Code_Walkthrough_Prompt.md` . Line numbers are current as of 2026-09-12.

File index

File	Lines	What breaks if you delete it
<code>common.py</code>	232	Everything. Holds the paths, the checksums, and the cleaning rules
<code>01_audit_routes.py</code>	323	The cleaning-funnel numbers and the 801-vs-803 justification
<code>02_extract_dir6.py</code>	143	The 229,421-row study set — the input to the whole simulation
<code>03_prepare_weather.py</code>	274	The NOAA tables. Weather would be unusable
<code>04_join_weather.py</code>	262	The rain flag on each stop event, and the feasibility verdict
<code>05_gtfs_gate.py</code>	126	The written record of why direction 6 is unlabelled
<code>run_all.py</code>	68	Convenience only — the five steps still run individually

Total runtime: 101 s. Measured: 42 + 54 + 2 + 2 + 1.

`common.py` — the shared plumbing

Purpose: holds everything the five steps have in common. Does no data processing itself.

L36–45 — imports

`hashlib` (checksums), `json` (config + audit files), `re` (one NOAA regex), `timedelta/timezone` , `Path` , `typing`, and `pandas` — needed only because `clean_masks()` annotates `DataFrame` types.

L48–74 — where everything lives

L52 `ROOT = Path(__file__).resolve().parents[2]`
WHAT the repository root
WHY `parents[2]` because this file is at <repo>/scripts/pipeline/common.py
WATCH move this file and every path in the project breaks. It is the anchor.

L54-58 `CONFIG_PATH, AUDIT_DIR, RAW_CAPMETRO_DIR, RAW_NOAA_DIR, PROCESSED_DIR`
WHY derived from `ROOT`, so nothing is tied to one machine

L62 `RAW_FULL_SNAPSHOT = RAW_CAPMETRO_DIR / "APC_Raw_July_2021_December_2021_full.csv"`
WHAT THE DATASET. This is the only place its name appears.
WATCH not in git (3.7 GB). Change this one line to move the file.

L70 `NOAA_LOCAL_STANDARD_TIME = timezone(timedelta(hours=-6), name="CST")`
WHY NOAA LCD timestamps are local STANDARD time with no daylight saving.
That is their documented convention, not an assumption.
WATCH get this wrong and every summer observation shifts an hour, smearing rain onto dry events for half the study window. Silently.

L74 `NUMBER_RE = re.compile(r"[-+]?[d+](?:\.\d+)?")`
WHY NOAA writes values like "0.05s" – a number with a quality letter

L80–93 — config and directories

L87 `return json.loads(CONFIG_PATH.read_text(encoding="utf-8"))`
WHY every tunable value lives in `config/texas_capmetro_801.json`, not in code: routes, direction codes, study window, NOAA stations, the 90-minute tolerance. Change the config, not the scripts.

L90-93 `ensure_dirs()` – `makedirs(parents=True, exist_ok=True)` on the five folders

L99–123 — checksums and safe writing

L101-105 `sha256_file()` – hashes in 1 MB blocks
WHY a 3.7 GB file cannot be read into memory at once
NOTE measured at ~3 s for the full snapshot; not a bottleneck

L111-115 `write_json()`
L111 writes to <name>.part first
L113 `json.dumps(..., indent=2, sort_keys=True)`
WHY `sort_keys` makes the output stable run-to-run, so a git diff shows real changes rather than reordered keys
L115 `temporary.replace(path)` – atomic rename
WHY an interrupted run leaves the old good file intact and the incomplete one obviously named. We never get a half-written audit file that looks valid.

L118-123 `write_text()` – same pattern for markdown evidence

L129–138 — `require_file()`

L131-137 raise `FileNotFoundError` with a message naming the missing file and pointing at the README's "Getting the data" section
WHY a new teammate's first run will fail – this pipeline reads archived data it does not download. A stack trace teaches them nothing.

L146–186 — `clean_masks()` ← the most important function in the project

The six cleaning rules, as four **cumulative** boolean masks.

```
L172 routes, directions = list(routes), list(directions)
    WHY .isin() needs a list; accepting any iterable and normalising here
        means callers can pass tuples

L174 on_route = frame["route_id"].isin(routes) # rule 1
L175 matching = on_route & (route_id == current_route_id) # rule 2
    WHY rule 2 drops records logged while a bus was being reassigned
        mid-trip – the route it is *assigned* to no longer matches the
        route it is *running*

L176-180 error_free = matching & (import_error=="") & (import_trip_error=="")
                                                # rules 3, 4
L181-185 clean = error_free & (bs_id != "") & direction_code_id.isin(directions)
                                                # rules 5, 6
    WHY bs_id "" means unknown stop – garage, deadhead, logon events

L186 return on_route, matching, error_free, clean
    WHY each mask is a superset of the next, so these four ARE the four
        bars on the cleaning-funnel slide
    WATCH step 1 uses all four; step 2 uses only `clean`. Defining them here
        once is what stops the funnel and the study set drifting apart.
```

L189–203 — `clean_where()`

Same six rules as an SQL-style string.

WATCH nothing executes this. `clean_masks()` is what runs. It exists so the audit JSONs and the manuscript can quote the cleaning rule as one readable line, and so anyone can re-run the identical filter against the public API. Do not delete it thinking it is dead.

L206–232 — NOAA quirk parsing

```
L211-221 first_value(row, *names) – first non-empty among several column names
    WHY NOAA changes capitalisation between files ("DATE" vs "Date")

L224-232 parse_flagged_number("0.05s") -> 0.05
    WHY LCD values carry trailing quality letters
```

Findings — `common.py`

- **Fixed 2026-09-12:** an orphaned `try/return float(value)/except` fragment sat at L209–212, left over from deleting `safe_float`. Because blank lines and comments do not close a Python block, it landed *inside* `clean_where()` after its `return` — unreachable, referencing an undefined name, and it parsed and ran fine. Removed.
- `pandas` is imported only for type annotations on `clean_masks`.

`01_audit_routes.py` — why 801, and the funnel

Feeds: nothing downstream. Produces evidence only.

L62–86 — constants

```
L64-69 NEEDED_COLUMNS - 19 of the 47
      WHY  reading 19 instead of 47 halves the read time (24 s vs 48 s)
L71  CHUNK_SIZE = 1_000_000
L74-86 ROWS - (label, stats-key) pairs defining the evidence table layout
```

L92–126 — `load_clean_rows()`

```
L96  levels = {route: {"all":0,"matching":0,"error_free":0} ...}
      WHAT  the funnel tallies, per route

L100-106 pd.read_csv(..., dtype=str, na_filter=False, chunksize=CHUNK_SIZE)
      WHY  dtype=str      the source types all 47 columns as text
      WHY  na_filter=False keeps blanks as "" instead of NaN, so the string
              comparisons in clean_masks behave predictably
              ("" == "0" is False, which is what we want)
      WATCH drop na_filter=False and the cleaning rules silently change meaning.

L112  on_route, matching, error_free, clean = clean_masks(chunk, candidates, ["4","6"])
L115-119 for each of the three upper levels, value_counts() by route, add in
L121  kept_chunks.append(chunk[clean])
L124  clean_df = pd.concat(kept_chunks, ignore_index=True)
      WHAT  ~832,455 rows – fits in memory comfortably
```

L132-193 — summarize()

```
L138-141 pd.to_numeric(..., errors="coerce") across nine columns
    WHY  unparseable values become NaN and are then skipped by mean/median,
         which reproduces the old hand-rolled "return None and exclude"
         behaviour exactly (verified against the committed evidence)

L145  df["service_day"] = df["transit_date_time"].str[:8]      # YYYYMMDD

L150-152 trip identity
    WHY  no single APC column is a reliable trip key. A trip is
         (service day, ext_trip_id), falling back to start-time + vehicle
         when ext_trip_id is blank.
    WATCH this is why distinct_trip_day_pairs is 24,943 and not something
         you can get from one column.

L155  usable_segment = (rev_seconds > 0) & (rev_distance > 0)
L158-162 good_gps = real coordinates AND quality_indicator between 3 and 6

L165  for route, rows in df.groupby("route_id", sort=True)
L169-192 the per-route output dict – see the traceability table below
```

L199-201 — percent_advantage()

The 801-vs-803 margin. Called three times; replaced three near-identical inline expressions.

Numbers this file produces

Number	Line	Appears in
547,616 / 468,689 all-route records	L119	Funnel slide, bar 2
524,728 / 445,369 matching	L119	Funnel slide, bar 3
480,452 / 403,254 error-free	L119	Funnel slide, bar 4
455,654 / 376,801 clean	L170	Funnel slide, bar 5
229,421 direction-6 events	L186	Funnel slide, bar 6 · everywhere
184 service days	L171	Slide 12
810,309 boardings	L173	Route-selection slide
11.414 mean max load	L175	Audit JSON
18.0 median dwell	L176	Audit JSON
98.136% GPS quality	L179	Slide 12
20.93% clean-event advantage	L201	Slide 11
29 stops per direction	L181	Slides 6, 9, 19

Findings — 01

- `alignments` (L174) is computed and published but **no downstream code reads it**. Confirmed by `grep`.
- The GPS-quality block (L158–162, L178–179) exists for one slide number.

02_extract_dir6.py — the study set

L65–123 — `extract_primary_subset()`

```
L67  require_file(RAW_FULL_SNAPSHOT, ...)
L69  destination = ROOT / config["apc"]["raw_output"]
      WHY    the output path comes from config, not hardcoded

L74  temporary = destination.with_suffix(".csv.part")
      WHY    an interrupted run must not leave a truncated study set that looks
            complete. This is the file everything downstream trusts.

L80-85 pd.read_csv(..., dtype=str, na_filter=False, chunksize=1_000_000)
      NOTE  no usecols here – all 47 columns are kept (see the file header)

L93  *, keep = clean_masks(chunk, ["801"], ["6"])
      WHAT  take only the last of the four cumulative masks
      WHY   step 2 needs the fully-clean rows; the intermediate funnel levels
            are step 1's job

L97-98 survivors.to_csv(temporary, mode="w" if first_chunk else "a",
                        header=first_chunk, index=False)
      WHY   append after the first chunk, and write the header only once
      WATCH if first_chunk handling is broken you get 10 header rows in the
            middle of the file

L103 temporary.replace(destination)  – atomic
```

Numbers this file produces

Number	Line	Appears in
229,421 rows	L96	The study set itself
47 columns	L89	Slide 7
SHA-256 8368412e...f7a23e21	L116	Provenance

Findings — 02

None. The file is almost entirely header.

03_prepare_weather.py — NOAA onto the bus clock

Deliberately still a plain Python loop. It handles NOAA's irregularities one at a time and vectorising would bury the reasoning.

normalize_weather_station() — the loop, per row

1. parse the timestamp
naive.replace(tzinfo=NOAA_LOCAL_STANDARD_TIME).astimezone(UTC)
WHY NOAA stamps are local standard time, no DST. A NOAA "15:00" in July is really 16:00 on an Austin clock.
WATCH THE highest-consequence line in the whole pipeline.
2. keep only 2021-07-01 .. 2021-12-31
3. pull out precipitation, present weather, temp, humidity, wind, visibility
via first_value() and parse_flagged_number()
4. rain_flag = 1 if ANY of:
precipitation > 0
precipitation field starts with "T" (trace)
present weather contains RA, DZ or TS
WHY deliberately inclusive – this is an EXPOSURE variable, not a rainfall measurement
5. deduplicate by instant, keeping the row with the most fields populated
WHY NOAA reports twice at the same instant (routine hourly + off-cycle special). merge_asof in step 4 must not have ties to break.

Numbers this file produces

Number	Appears in
6,484 Camp Mabry observations, 584 rain-flagged	Slide 16
5,472 Bergstrom observations, 480 rain-flagged	Audit JSON

Findings — 03

- The header still lists the NOAA URLs as inputs; they are now only used to derive the local filename (station["url"].rsplit("/", 1)[-1]). Harmless but slightly misleading.

L78–104 — load_apc_events()

```
L89-91 read only apc_date_time, rev_seconds, rev_distance
L92  pd.to_datetime(..., format="%Y%m%d%H%M%S", errors="coerce")

L96  first_pass = tz_localize(tz, ambiguous=True, nonexistent="shift_forward")
L97  second_pass = tz_localize(tz, ambiguous=False, nonexistent="shift_forward")
L98  dst_ambiguous = int((first_pass != second_pass).sum())
    WHAT counts timestamps inside the repeated fall-back hour
    WHY  on 7 Nov 2021 the hour 01:00-02:00 happened twice in Austin. A bare
        APC timestamp in that hour is genuinely ambiguous. Localizing both
        ways and comparing finds them; we then use the first pass and
        REPORT the count rather than silently guessing.
    NOTE the count is 0 for this dataset – no dir-6 events fall in that hour

L100 events["event_time"] = first_pass.dt.tz_convert("UTC")
    WHY  matching happens on UTC instants, which are unambiguous
```

L107–115 — load_weather()

```
sorted by obs_time – merge_asof REQUIRES both sides sorted on the key
WATCH remove the sort and merge_asof raises, or worse, silently mismatches
```

L118–136 — join_nearest()

```
L124-132 pd.merge_asof(events, weather, left_on="event_time",
                        direction="nearest", tolerance=tolerance)
    WHAT for each event take the closest observation in time, within 90 min
    WHY  this one call replaced ~40 lines of hand-rolled bisect search
    WATCH unmatched events get NaN – that is how coverage is measured

L133-135 delta_min = |event_time - obs_time| in minutes
```

```
L151-152 two joins: Camp Mabry (primary), then Bergstrom (sensitivity)
    WHY the second is the cross-check that produces the 94.0% agreement

L161 p95 = np.sort(deltas)[ceil(0.95 * n) - 1]
    WATCH deliberately the plain order statistic, NOT pandas .quantile(),
    which interpolates and would give a different number. This matches
    the original audit.

L182 usable = primary[(rev_seconds > 0) & (rev_distance > 0)]
L183-184 wet / dry split for the descriptive medians
    WATCH 204 s vs 212 s is DESCRIPTIVE ONLY. Austin rain is convective and
    clusters in the afternoon, so part of that gap is rush hour. The
    warning is written into the output JSON so it cannot be quoted out
    of context.

L188 feasible = coverage >= 0.95 and rain_rows >= 1000
    WHY the rule was declared BEFORE seeing the result
```

Numbers this file produces

Number	Line	Appears in
100% join coverage	L186	Slide 16
11,804 rain-exposed events	L165	Slides 12, 16
12.667 min median join gap	L199	Slide 16
27.933 min p95	L161	Defense material
215,705 / 229,421 = 94.0% station agreement	L166	Slide 16
204 s dry vs 212 s rain	L183–184	Slide 16

Findings — 04

None.

05_gtfs_gate.py — the direction-label gate

No data processing. Reads the gtfs block of the config and writes the document recording that direction 6 may not be called "southbound" until a 2021 GTFS snapshot is verified.

Findings — 05

- The gate text is now **out of date in substance**: the 2021 GTFS snapshot has been confirmed not publicly archived, and direction 6 has been corroborated as southbound via the 2026 feed overlay. The wording still says "not yet verified". Update the config's gtfs block, not this script.

`run_all.py`

Runs the five steps in order via `subprocess`, stopping on the first non-zero exit. `STEPS` at L38–44 is the list; order matters only because step 4 needs the outputs of steps 2 and 3.

Consolidated findings

	Where	Status
Orphaned <code>try/except</code> fragment inside <code>clean_where()</code>	<code>common.py:209–212</code>	Fixed 2026-09-12
GTFS gate wording contradicts what we now know	05 / config <code>gtfs</code> block	Open — update the config
<code>alightings</code> computed and published, never read	01:174	Open — harmless
03 header lists NOAA URLs as inputs; only the filename is used	03 header	Open — cosmetic
<code>clean_where()</code> has no callers by design	<code>common.py:189</code>	Not a defect. Do not delete

The traceability rule

Every number in the deck traces to a line in this document, and from there to a file in `data/audit/texas_capmetro/`. If you need a figure, take it from the audit JSON or regenerate it — never retype it from a slide.